

Where the Seconds Go:

A Host-Memory Law and a Speed Limit for Mixture-of-Experts Decode on One Consumer GPU

Harshith Kantamneni
Independent Researcher

Abstract

A Mixture-of-Experts (MoE) model too large for one consumer GPU keeps most of its experts in host memory, and every decoded token waits while some of them are read. We ask where each token’s time goes on such a machine and how much of it any system could remove. **(1) A host-memory law.** Time per token is a fixed GPU term plus the expert bytes read from host memory, divided by the rate of the read paths in use (CPU, PCIe, or both at once). Written on each machine from its own bandwidth probe before any model run, it predicted 29 configurations of our cache on 7 machines with 3.7% median error. It fails in three cases, and we explain each. **(2) A speed limit.** The per-layer Belady optimum with bypass, on routing traces of the models’ own text for the measured prompts, together with the machine’s highest measured host read rate, bounds any exact-routing system at a given GPU expert budget. The fastest system we measured reaches 26–43% of it. Decomposing the rest, missing foresight is the largest or tied-largest cost (17–27% of each token), with serialised host reads close behind. **(3) A system the law implies.** We built an expert cache into llama.cpp. It runs misses on the CPU and sets, per machine, how many of a layer’s missed experts to copy over PCIe instead. At equal GPU memory on one RTX 5090 machine it decodes gpt-oss-120b 15–29% faster than FreeToken and Qwen3-30B-A3B 3–15% faster, and runs 2.0–3.8 $\times$ as fast as llama.cpp; its lead over FreeToken holds over 2,048-token outputs. **(4) Foresight.** The optimum reads 22–56% fewer experts from host memory than the best online policy. The lookahead that closes half of that gap grows as $\approx 0.51 (C/k)^{1.39}$ tokens across 9 models. Every prediction was committed before the measurement it predicts.

1 Introduction

Open-weight Mixture-of-Experts models now carry 30–120B parameters and activate a few billion per token; a desktop GPU has 24–32GB. Running such a model at batch 1 therefore means keeping most experts in host DRAM, and each decoded token then reads its routed experts from one of three places: GPU memory, host memory by the CPU that runs the expert, or host memory across

PCIe into a GPU slot [14, 10]. A line of systems has built on this: GPU expert caches with demand fetch [6, 32, 25], CPU execution of misses [16, 5, 39, 12, 38, 41], per-miss choices between the two [34], and expert prediction [13, 40, 30]. Each reports a speed-up over a baseline of its choosing. None states how fast its machine *could* decode the model within the same GPU memory, which host parameter decides whether a miss should be copied or computed where it is, or how much of the remaining time any design could remove.

We answer these questions on consumer hardware (RTX 5090 hosts with desktop CPUs, rented on Vast) with two models, gpt-oss-120b [22] and Qwen3-30B-A3B [33]. The method is Hoeffler and Belli’s [11]: a performance model with bounds, statistically sound comparisons, and every prediction committed to a public repository before the machine it concerns was started (Appendices A and B).

Contributions.

- **A host-memory law** (Section 3): time per token = $G + \max(X_c/B_c, X_p/B_p, (X_c+X_p)/B_{cp})$, with host bytes X from the engine’s own counters and read rates B from a probe of the machine. Predicted blind on 7 machines, its median error is 3.7% over 29 configurations; the three places it fails each point at something the law leaves out.
- **A speed limit, and where the seconds go** (Section 4): a bound on any exact-routing system with C expert slots per layer, from exact routing traces and measured bandwidths, reported next to every measured speed (Table 1), and a decomposition of the gap into overlap, foresight, policy, GPU efficiency and host work.
- **A system that follows from the law** (Section 5): a llama.cpp expert cache whose miss path is chosen per machine from the law. It leads FreeToken [34] at every budget we measured, runs 2.0–3.8 $\times$ llama.cpp at equal GPU memory, and an ablation says which parts matter.
- **The value of foresight** (Section 6): how many host reads knowing future routing would save, and how far ahead one must know it, across nine models.

What we do not claim. Running missed experts on the CPU [16, 5], caching hot experts in GPU memory with CPU-executed misses [39, 12, 18], and splitting a layer’s misses between the CPU and PCIe [34, 41] are all published, as is matching one offloaded configuration’s decode speed to bytes per token over bandwidth [26]. Our system uses these ideas; what is new is the law that says when each path pays on a given machine, the limit it is measured against, and the measurements.

2 Setting and Method

Workload. Single-request decode. gpt-oss-120b (MXFP4 experts; 36 MoE layers of 128 experts, top-4, 13.25 MB per expert) and Qwen3-30B-A3B in BF16 (48 layers of 128 experts, top-8, 9.44 MB per expert). Prompts are the 30 AIME-25 problems in FreeToken’s own benchmark format; each system decodes 256 tokens per problem, greedily, after one warm-up request on a fixed held-out text, in one session in which the cache carries over between problems. One client measures every system with FreeToken’s own metric, (tokens−1) over the time between the first and last token’s arrival.

Machines and probes. RTX 5090 (32 GB, 1.79 TB/s) hosts with desktop CPUs, rented by the hour. Each job first runs a probe (concur) that reads host memory from t CPU threads, over PCIe with zero-copy kernels and with the copy engine, and from both at once; FreeToken’s own probe (ft bench bw) runs too. We call the CPU rate at the helper thread count B_c , the zero-copy PCIe rate B_p and the combined rate B_{cp} . GPU clocks cannot be locked on these rentals; we report the power limit of each card.

Equal memory. llama.cpp keeps whole layers’ experts on the GPU (--n-cpu-moe n), our cache keeps C experts of every layer, and FreeToken takes a cache rate; we match the number of experts on the GPU (within 2.5%) and report measured VRAM. FreeToken preallocates 28.4–30.1 GiB whatever its rate.

Statistics and protocol. Ratios are of mean speeds, paired by problem, with 95% percentile intervals from 10,000 bootstrap resamples. Where a system has variants (our FETCH tables, FreeToken’s offload and hybrid back-ends), a first launch picks each system’s faster variant per budget and a second launch, which the comparison uses, runs only those (one row of Table 1 has no second launch). Every job’s predictions are in the header of its script, committed and pushed before the machine starts.

3 A Host-Memory Law

With experts partly in host memory, a decode step does two kinds of work: GPU work that every token needs

(attention, dense projections, resident experts, the output head, launches), and reads of the experts that are not resident. The second is limited by host memory. The CPU reads X_c bytes per token (the experts it runs), PCIe reads X_p bytes (the experts copied to the GPU, whether on demand or ahead of time), and when both run at once they share the DRAM:

$$T = G + \max\left(\frac{X_c}{B_c}, \frac{X_p}{B_p}, \frac{X_c + X_p}{B_{cp}}\right). \quad (1)$$

X_c and X_p come from the engine’s counters of misses, fetches and admissions, which do not depend on the machine. B_c , B_p and B_{cp} come from the machine’s probe. G is one constant per engine and model, fitted once on one host (4.82 ms for our cache on gpt-oss-120b, 5.06 ms for llama.cpp) and then frozen in the repository.

Blind test. On every later machine the job wrote the law’s prediction from its own probe before any model ran (Table 2). Over 29 configurations of our cache on 7 machines (33 measurements; one machine ran twice) the median error is 3.7%; without the 12-channel EPYC 9655 it is 3.3% over 24 configurations (90th percentile 7.5%). llama.cpp --n-cpu-moe is predicted within 4% on the Ryzen hosts, including two Qwen3 configurations predicted on the headline machine (+3.6%, +4.0%; for Qwen3, $G=4.3$ ms, the mean residual of three llama.cpp runs on another host, job 078).

Where it fails. Each failure names a term the law leaves out. (i) Copying misses on demand (FETCH) at 25% is over-predicted by about 12% on the two hosts with the slowest PCIe links, the Core Ultra 7 and the EPYC 7352: the law has no per-layer copy latency. (ii) llama.cpp on a Core Ultra 7 with 8 performance and 16 efficiency cores runs at 46% of the law, because it splits every matrix evenly across its threads; our helpers claim work dynamically and stay on the law on the same machine. (iii) On a 12-channel EPYC 9655 (559 GB/s) host memory stops being the limit: a layer with one or two CPU misses takes 80–200 μ s, a latency floor of the per-layer hand-off.

What the law says. Every mechanism in this design space acts only through the bytes read from host memory per token and the number of read paths in use. A second path adds bandwidth on every host we measured but the 12-channel EPYC (B_{cp} against $\max(B_c, B_p)$): 8–17% on the Ryzen hosts, 1% on the Core Ultra, 33% on the Zen 2 EPYC), and that is the whole gain available to fetching, prefetching or splitting misses between the CPU and PCIe. Prefetching never reduces the bytes. And the “host lottery” is the DRAM: two rentals of the same CPU differed by 19–22% in speed, which is 62 against 47 GB/s of host bandwidth.

Table 1: Decode speed at equal GPU expert memory on one machine (RTX 5090 + Ryzen 9 9950X3D; host memory read by the CPU at 71.6 GB/s and over PCIe at 53.2 GB/s). 30 AIME-25 problems, first 256 decode tokens, greedy, session. Ours uses the FETCH split computed from this machine’s bandwidth probe; FreeToken uses its faster backend per budget. Ratios are of mean speeds, paired by problem, with 95% bootstrap intervals; the comparison launch is separate from the launch that picked each system’s variant. *Speed limit*: the fastest any exact-routing system with the same slots per layer can decode on this machine (Section 4). *All in VRAM*: stock llama.cpp with every weight on an RTX PRO 6000 (same memory bandwidth as the RTX 5090, 96 GB). [†]Launch 1 for both systems (job 080): FreeToken’s backend was picked on the launch it is scored on, which favours it, and the row has no confirmation launch.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Speed limit	Ours, % of limit	All in VRAM
gpt-oss-120b	11%	34.9	54.0 (hybrid)	69.9	1.294 [1.278, 1.312]	2.00×	169	41%	255
gpt-oss-120b	25%	40.0	85.6 (hybrid)	109.2	1.275 [1.254, 1.295]	2.73×	427	26%	255
gpt-oss-120b	40%	47.7	132.2 (offload)	152.5	1.154 [1.134, 1.172]	3.20×	519	29%	255
Qwen3-30B-A3B	12.5%	19.5	38.7 (hybrid)	40.0	1.032 [1.022, 1.042]	2.05×	92	43%	178
Qwen3-30B-A3B	25%	22.1	54.7 (hybrid)	63.1	1.153 [1.134, 1.173] [†]	2.86×	209	30%	178
Qwen3-30B-A3B	43.75%	28.4	103.1 (offload)	108.2	1.049 [1.036, 1.064]	3.81×	308	35%	178

Table 2: The law predicted blind: each machine wrote its prediction from its own bandwidth probe, with constants frozen in the public repository, before any model run. Error of predicted over measured speed, gpt-oss-120b (cache rows) and llama.cpp --n-cpu-moe (last column, %).

Host CPU	Job	n	Median err %	Max err %	llama.cpp err %
Core Ultra 7 270K (8P+16E)	069c	5	1.5	11.9	+119
EPYC 7352 (Zen 2)	069c	5	3.2	11.7	+10
EPYC 9655 (12-ch. DDR5)	069c	5	18.7	31.9	+16
Ryzen 9 7900	072	5	4.9	8.2	+4
Ryzen 9 9950X3D (A)	073	8	1.6	4.5	–
Ryzen 9 7950X	074	1	4.0	4.0	–
Ryzen 9 9950X	076	4	2.5	5.0	–
Ryzen 9 9950X3D (B)	082	0	–	–	+4, +4

4 The Speed Limit, and Where the Seconds Go

Bound. Consider any system that executes the model’s exact routing with at most C experts of each layer resident on the GPU. Per token, each of the Lk routed experts is either served by a resident copy (the GPU reads it from VRAM) or not, in which case its S bytes are read from host memory once more: by the CPU that runs it, or by the copy that brings it into a slot. Let R^* be the reads per token of Belady’s MIN with bypass [2, 7, 36] applied per layer to the routing trace, the fewest reads of any policy, even one that knows the future. If c experts per token run on the CPU, host memory supplies at least $\max(R^*, c)S$ bytes and the GPU reads the dense bytes D plus $(Lk - c)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq c \leq Lk} \max \left(\frac{D + (Lk - c)S}{B_{\text{gpu}}}, \frac{\max(R^*, c)S}{B_{\text{host}}} \right), \quad (2)$$

with B_{gpu} the GPU’s datasheet rate and B_{host} the highest host read rate the probe measured by any method

(87.5 GB/s on the headline machine). The bound assumes that an expert run on the CPU is read from DRAM (the reuse distance of an expert is hundreds of MB, beyond any CPU cache), that computation runs at memory speed, and that the two read paths can share B_{host} freely; it drops all latencies and starts from an empty cache. B_{host} is the highest rate our probe reached, not a hardware peak, so a system that reads host memory faster than the probe could exceed the limit. It can exceed the all-in-VRAM speed, because the CPU’s bandwidth adds to the GPU’s. The traces are the models’ own greedy text on the 30 AIME-25 prompts (generated all on the GPU by stock llama.cpp), templated and tokenised exactly as the server does, with routing recorded teacher-forced by our llama.cpp instrumentation (jobs 084b and 084c); each system’s own greedy text drifts from it within the first few hundred characters on gpt-oss (Section 5).

How far measured systems are from it. Table 1 reports the limit next to every measured speed. Our system reaches 26–43% of it, FreeToken less. At the largest budgets the limit exceeds the all-in-VRAM speed at datasheet rate, because the CPU can run some resident experts in parallel with the GPU; there the limit is set by the machine’s aggregate bandwidth, not by misses. Even with every weight in VRAM, stock llama.cpp reaches only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate at batch 1, and our cache with every expert in its slots matches or beats it (0.997× and 1.047×): the cache costs nothing when the model fits. The traces represent the measured runs: replayed through a simulator of our deployed policy, they give hit rates within 1.4 points of the engine’s own counters on gpt-oss and within 0.6–5.3 points on Qwen3; the simulator omits FETCH, whose immediate admissions raise the engine’s hit rate most at the smallest budget.

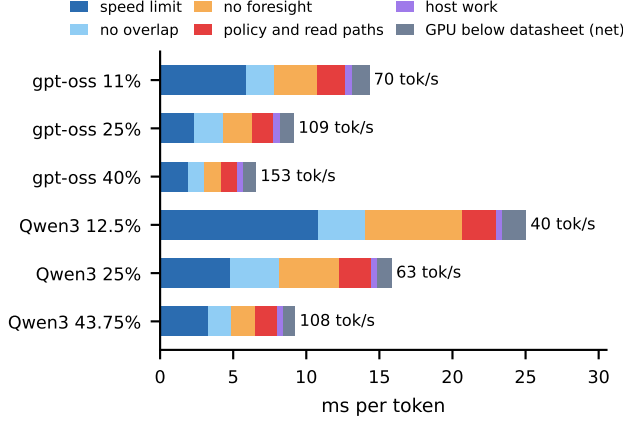


Figure 1: Where the seconds go: for every cell of Table 1, from the speed limit to measured time per token, one cause at a time. Steps: GPU and host work in sequence instead of overlapped; the best online policy’s reads instead of the optimum’s; the engine’s own reads and read paths, through Eq. (1); per-token host work, from the engine’s timer; and the rest, mostly GPU kernels below datasheet rate net of the overlap the engine achieves.

Decomposition. Figure 1 adds one cost at a time to the limit, in the order listed, using only measured or traced quantities; another order would attribute shared time differently (costed with overlap, for instance, foresight is worth less at the largest budgets, where the limit is set by aggregate bandwidth). Missing foresight, the best online policy’s reads instead of the optimum’s, is the largest or tied-largest step in all 6 cells and costs 17–27% of each token. Running host reads and GPU work in sequence costs 13–22%, our policy and read paths beyond the best online policy 9–17%, per-token host work 2–7%, and GPU kernels below datasheet rate, net of the overlap the engine achieves, 6–13%. At Qwen3 12.5% our cache already reads as few experts per token as the best online policy (165); its policy step there is only its read paths running below the machine’s highest rate. Foresight is the one term that no engineering of read paths can recover (Section 6).

One token, profiled. On Qwen3 at 43.75% with our earlier fixed FETCH table (job 079, listing A), Nsight Systems splits a token of our cache (FreeToken) into 4.93 (5.20) ms of expert copies over PCIe, 2.29 (2.21) of resident-expert products, 1.65 (1.34) of attention projections, 0.49 (0.37) of attention, 0.38 of output head and 0.97 (0.40) of norms, small kernels and cache control; our host-side time is 0.27 ms (job 079). Neither system overlaps its PCIe copies with GPU compute; doing so would save 5–10%. Our extra 0.5 ms of GPU work comes from unfused batch-1 kernels: we launch 1.6× as many.

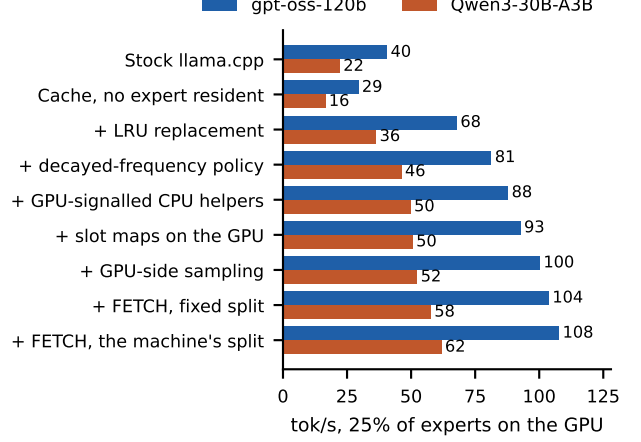


Figure 2: Ablation at 25% of experts on the GPU (15 AIME-25 problems, 256 tokens; job 082). Each step adds one component; every step after the second is faster than the one before, with the 95% interval above 1.

5 A System That Follows From the Law

Design. Our cache is a 3,700-line patch to llama.cpp. Each MoE layer has C GPU slots. Routing is decided on the GPU; resident experts run there, and each miss either runs on CPU helper threads, which the GPU signals through a mailbox in mapped memory, or is copied into a slot and run on the GPU (FETCH). Admission follows a decayed-frequency score (half-life 16 tokens, an admission margin $\kappa=1$) and copies in the background. Sampling runs on the GPU. The one per-machine decision is the FETCH table $f(n)$: of a layer’s n missed experts, how many to copy. A per-layer form of Eq. (1) gives it,

$$T(n, f) = \max\left(\frac{fS}{B_p} + g, [n > f](L_c + \frac{(n-f)S}{B_c}), [0 < f < n]\frac{nS}{B_{cp}}\right), \quad (3)$$

where g is the GPU’s expert work per layer ($48 \mu s$ for Qwen3, $37 \mu s$ for gpt-oss, from profiles) and $L_c=20 \mu s$ the hand-off; the table picks the f with the smallest T and is computed from the probe when the job starts.

Where the gain comes from. Figure 2 builds the system up from stock llama.cpp. The cache’s host-driven CPU path with no expert resident is slower than stock ($0.72\times$, $0.74\times$), which pins whole layers and runs the rest on the CPU without per-layer hand-offs. Filling the slots, here with LRU replacement, is the largest step ($2.31\times$ and $2.21\times$), and the decayed-frequency policy adds 20% and 27%. A static cache of each layer’s most requested experts, profiled on other text, [pend.] (job 087; the ablation’s original static step loaded no experts). The mailbox, GPU slot maps, GPU sampling and FETCH add 2–10% each. The full system runs $2.66\times$ (gpt-oss) and $2.80\times$ (Qwen3)

Table 3: FETCH tables at 43.75% on Qwen3-30B-A3B (launch 1): entry n is how many of a layer’s n missed experts are copied over PCIe; the rest run on the CPU. Ratio to the fixed table, paired, 95% CI.

Table	$f(1..8)$	tok/s	vs fixed
Lite	0,0,1,1,1,2,2,3,3	109.0	1.057 [1.052, 1.062]
Law	0,0,1,1,2,3,3,4,5	108.0	1.048 [1.041, 1.055]
Law, fetch a single miss	0,1,1,1,2,3,3,4,5	104.1	1.010 [1.007, 1.012]
Fixed	0,1,1,2,3,3,4,5,6	103.1	1
None (CPU only)	0,...,0	100.7	0.977 [0.970, 0.984]
All (fetch every miss)	0,1,...,8	98.7	0.957 [0.953, 0.962]

stock llama.cpp.

Head to head. Table 1 compares the three systems at six budgets on one machine. Ours leads FreeToken at every budget, by 15–29% on gpt-oss and 3–15% on Qwen3, and runs 2.0–3.8× llama.cpp. The lead is largest when many experts miss and the CPU path carries them (gpt-oss at 11–25%). It is smallest at Qwen3 12.5%, where our cache already reads host memory at 62 GB/s, 80% of the machine’s combined rate, and at Qwen3 43.75%, where few experts miss and FreeToken’s fused GPU kernels count. On FreeToken’s own headline model, Qwen3.6-35B-A3B in BF16, [pend.] (job 086).

Choosing the split per machine. On the headline machine, the law’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0% (launch 1). Table 3 shows why at one budget: a single missed expert should run on the CPU (about 155 μ s, hidden behind the GPU’s other experts) rather than be copied (about 225 μ s, on the critical path); that entry alone is worth 3.7%. Fetching everything, FreeToken’s offload strategy expressed in our engine, is the worst table, and running everything on the CPU the second worst. The law’s table was within 1% of the best of five alternatives on launch 1 and tied with it on launch 2. Given each run’s measured misses, Eq. (3) ranks the tables correctly except law against lite, which are within 1%, and overstates their differences about 1.8×. On a contrasting machine, an RTX 5090 in a PCIe 4.0-class slot next to a Core i9-14900K, [pend.] (job 085).

Long outputs. Our lead does not come from the first 256 tokens. On gpt-oss, on ten held-out hard problems (AIME 2022, problems 11–15 of both exams) that neither system has finished reasoning about after 2,048 tokens (at most 1 of 10 reach an answer), ours runs 1.36 [1.34, 1.38], 1.32 [1.30, 1.34] and 1.19 [1.17, 1.21] times FreeToken at 11, 25 and 40% over all 2,048 tokens, and 1.37 [1.34, 1.39], 1.33 [1.31, 1.35] and 1.19 [1.17, 1.22] over tokens 257–2,048 (job 083, listing A); both engines stream one event per token. On Qwen3, over tokens 257–2,048 of ten held-out MATH-500 problems, ours runs 1.043 [1.020, 1.070], 1.056 [1.036, 1.077] and 1.054 [1.025, 1.086] times FreeToken at

12.5, 25 and 43.75%. Both leads are larger than over the same runs’ first 256 tokens. Turning off FreeToken’s prefill overlap, which lends it cache slots during each prompt, changes its overall speed by 1% or less, though its rate over the first 256 tokens drops 5–6%.

Other systems. Two newer llama.cpp-based systems, Pipelined Sharding [28] and leloch’s expert cache [17], run gpt-oss-120b at 1.28–1.45× stock llama.cpp; ours ran 1.42–2.34× the better of them (job 077). With BF16 weights on a CPU without AMX, KTransformers [5] runs Qwen3 at llama.cpp’s speed (job 078); that is not its best setting, which uses AMX and INT4/INT8 weights.

Outputs. Teacher-forced on fixed text, our system picks the same next token as stock llama.cpp on 98.6% (gpt-oss) and 99.3% (Qwen3) of steps, against 99.6% and 99.9% for stock llama.cpp with a different layer placement; the loss changes by -0.22% and +0.31%. The differences are floating-point order, not quality, and explain why free-running greedy text diverges early on gpt-oss.

6 The Value of Foresight

The largest step in Fig. 1, and the one no read path can close, is the policy’s ignorance of future routing. In the decomposition it accounts for 17–27% of each token. How much foresight would recover it? We replay the models’ own sampled text for 9 MoE models [21, 33, 22], with the cache carried across conversations, at 12.5, 25 and 50% of experts per layer (26 model–budget points), and compare the best online policy with Belady’s optimum and with policies that see the next W tokens’ routing (Belady within the window, decayed frequency beyond it; the family of [7]):

- The optimum reads 22–56% fewer experts from host memory than the best online policy (median 39%).
- The lookahead needed scales with the cache’s turnover time C/k , slots per layer over experts per token: half of the gap closes at $W_{50} \approx 0.51(C/k)^{1.39}$ tokens ($r=0.975$ in log–log over 26 points), and 90% needs a median of $2.3 C/k$. The exponent is the same on greedy own text (1.38) and on teacher-forced dataset text (1.43).
- Small caches ($C/k \leq 2$) recover half of the gap with one or two tokens of foresight; gpt-oss-120b at 25% ($C/k=8$) needs about 10 tokens, and at 50% about 34.
- Predicting the next layer’s experts, the usual form of prediction [13, 40], gives no cross-token foresight: it can raise the hit rate by prefetching but never lowers host bytes, which is what we measured when we built it.

Zhang [37] attributes most of the online-to-optimal gap to not knowing which resident expert is used furthest in the future; the horizon’s scaling with C/k and its price in seconds are new here. They say where foresight must come from: at the budgets of a consumer GPU, a few to tens of tokens of routing ahead, which speculative decoding or a router decoupled from the backbone [3] could plausibly provide.

7 Related Work

Offloading systems. Demand-paging systems cache experts per layer and copy misses over PCIe [6, 32, 25, 27]. CPU-execution systems run non-resident experts on the host [16, 5, 39, 38, 29]; a set-associative GPU cache with CPU-executed misses [12] and a llama.cpp fork [17, 18] are the closest designs to ours, and FreeToken [34] and DALI [41] split each layer’s misses between CPU and PCIe, FreeToken from a closed-form bandwidth model. Pipelined Sharding [28] plans sub-layer placement under a VRAM budget. Predictors guess upcoming experts from the previous layer’s state [13, 40], several steps ahead [30], or a batch’s routes across layers [8]; Read-ME [3] makes routing known in advance by decoupling the router and then caches in Belady’s manner. Batched-throughput systems [4, 35] and SSD tiers [19, 20] are outside our single-request, host-DRAM scope.

Models and bounds. FreeToken’s split and our Eq. (3) are both bandwidth models of one layer; Eq. (1) is a whole-token model tested blind across machines. MoE-CAP [15] measures bandwidth utilisation against a single peak; Qiu et al. [23] predict llama.cpp throughput from GGUF meta-data for models held entirely in GPU or unified memory; Suram [26] matches one offloaded configuration to bytes per token over bandwidth; Si et al. [24] derive when a copy pays off. The roofline [31] is the single-memory ancestor of our bound, and Hoeffler and Belli’s rule of showing upper bounds [11] its motivation. Zhang [37] proposes a trace-level evaluation contract for expert caching and decomposes the online-to-optimal gap in cache blocks; SpecMD [9] benchmarks cache and prefetch policies across hardware.

Lookahead caching. Limited lookahead in paging goes back to Albers [1]; Hasslinger et al. [7] combine Belady within a window with LRU or LFU beyond it for web caches, the policy family we use to measure the horizon.

8 Limitations

- **Hardware.** One GPU model (RTX 5090; an RTX PRO 6000 for all-in-VRAM runs) and desktop Ryzen and Intel hosts, rented; GPU clocks could not be

locked and power limits differed (400–575 W). A 12-channel server CPU breaks the law’s bandwidth assumption (Section 3).

- **Workload.** Single-request decode; AIME prompts, which were also used while developing the system; greedy decoding. Long outputs are measured on held-out problems. Batch sizes above one and long contexts are not.
- **Competitor settings.** FreeToken ran its faster backend per budget with its own bandwidth calibration; KTransformers ran in a setting that is not its best. Each comparison uses one confirmation launch, except Qwen3 at 25%, which is from the selection launch for both systems.
- **The bound.** Exact routing only: expert skipping, substitution or deferral change the model and fall outside it. The bound drops latencies and is loose where the GPU term dominates.
- **Constants.** G , g and L_c are fitted once and frozen; the per-layer model overstates differences between FETCH tables about $1.8\times$.

9 Conclusion

On a consumer machine, MoE decode time is a fixed GPU term plus host-memory bytes over the rate of the paths that read them. That law, predicted blind, is accurate to a few percent on desktop hosts; it tells a system which path to use for a miss on a given machine; and with the optimum’s bytes it gives a speed limit that shows how much of each token remains to be won, and by what. Most of what remains, at every budget we measured, is missing foresight and host reads serialised with GPU work.

Acknowledgments. Parts of the code, the experiments and the text were drafted with the help of an AI system (Claude, Anthropic). The author reviewed the work and takes responsibility for its content.

References

- [1] Susanne Albers. On the influence of lookahead in competitive paging algorithms. *Algorithmica*, 18(3):283–305, 1997.
- [2] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [3] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 116126–116148, 2024. arXiv:2410.19123.

- [4] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [5] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.
- [6] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [7] Gerhard Hasslinger, Juho Heikkinen, Konstantinos Ntougias, Frank Hasslinger, and Oliver Hohfeld. Optimum caching versus LRU and LFU: Comparison and combined limited look-ahead strategies. In *Proceedings of the 16th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, pages 1–6, 2018.
- [8] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling. *arXiv preprint arXiv:2410.17954*, 2024. Accepted at DAC 2026 (per arXiv v2).
- [9] Duc Hoang, Ajay Jaiswal, Mohammad Samragh, and Minsik Cho. SpecMD: A comprehensive study on speculative expert prefetching. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03921.
- [10] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [11] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [12] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [13] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.
- [14] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [15] Yinsicheng Jiang, Yao Fu, Yeqi Huang, et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- [16] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [17] leloch. MoE expert cache for llama.cpp: VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. <https://github.com/leloch/llama.cpp/tree/moe-cache-v2-pr>, 2026. Branch moe-cache-v2-pr, commit e3096b0 (6 August 2026). Proposed upstream as llama.cpp pull request #24524 and RFC discussion #24528; not merged. Accessed September 2026.
- [18] leloch. RFC: MoE expert cache, VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. llama.cpp discussion #24528, <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Posted 12 June 2026; accessed September 2026.
- [19] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.
- [20] Yu Lin, Yiming Wang, Runyuan Cai, Hanze Liu, and Xiaodong Zeng. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- [21] Niklas Muennighoff, Luca Soldaini, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- [22] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [23] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [24] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [25] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [26] Shriniwas Ramesh Suram. Cacheable by design? training mixture-of-experts routers for locality against the edge memory-bandwidth wall: A pre-registered negative result, with a systems measurement study. *arXiv preprint arXiv:2608.18261*, 2026.
- [27] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [28] Aditya Ukarande, Deep Shekhar, Marc Blackstein, and Ram Rangan. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 8, 2026. Industry track. arXiv:2604.26334. Artifact: <https://github.com/deepshnv/pipeshard-mlsys26-ae>, tag v2.0.3-mlsys26.
- [29] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [30] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.

- [31] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [32] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [33] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [34] Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [35] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [36] Lei Zhang, Reza Karimi, Irfan Ahmad, and Ymir Vigfusson. Optimal data placement for heterogeneous cache, memory, and storage systems. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (PO-MACS)*, 4(1):6:1–6:27, 2020.
- [37] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [38] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference. In *Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. *arXiv:2501.10375*.
- [39] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. *arXiv:2504.05897*.
- [40] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.
- [41] Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. DALI: A workload-aware offloading framework for efficient MoE inference on local PCs. *arXiv preprint arXiv:2602.03495*, 2026.

A Scientific benchmarking checklist

How this paper follows the twelve rules of Hoefer and Belli [11].

Rule	Where and how
1. Report the base case of a speed-up and its absolute performance.	Every ratio in Table 1 is next to both absolute speeds; llama.cpp is configured with per-layer expert offload at equal expert memory, the setting its users run.
2. Explain subsets of benchmarks or resources.	Two models chosen for size and precision (MXFP4, BF16), a third that is FreeToken’s own headline model; budgets span what fits a 32 GB card; the 43.75% budget is omitted where it does not fit. All 30 AIME-25 problems are used.
3. Arithmetic mean for costs, harmonic mean for rates.	Every problem decodes the same number of tokens, so the harmonic mean of rates is total tokens over total time; ratios of total time agree with the reported ratios of mean rates within 0.003.
4. Avoid summarising ratios.	Ratios are computed from mean speeds, never averaged across budgets or models; ranges list the per-budget values.
5. Report whether measurements are deterministic; give confidence intervals.	Decode is greedy and deterministic per engine; timing is not. Every comparison has a 95% interval.
6. Do not assume normality.	Intervals are bootstrap percentile intervals (10,000 resamples, paired by problem); no distributional assumption.
7. Compare nondeterministic data soundly.	Paired bootstrap of the ratio of means; the variant a system is scored with is picked on a separate launch (except the Qwen3 25% row of Table 1).
8. Check that central tendency is the right summary.	Mean speed over 256 tokens is FreeToken’s own metric; per-token p50 and p99 latencies are recorded for every problem in the released rows.
9. Document all factors and the setup.	Machines, probes, power limits, software commits (llama.cpp 4da6337, FreeToken 0d652e7), flags and job scripts are released; Appendix C.
10. Report measurement, synchronisation and summarisation.	One client times every system from streamed token arrivals; no server-side timers enter a comparison (Section 2).
11. Show upper performance bounds.	The speed limit (Section 4) and all-in-VRAM speed are columns of Table 1.
12. Plot enough to interpret; connect points only for trends.	Figures 1 and 2 are bar and step charts; no interpolation.

B Prediction log

Each job’s predictions are in the header of its script on the public `gpu` branch, committed and pushed before the machine started; git history orders every prediction before its measurement. Outcome notes with the statistics are in `prereg/`. Of the 33 predictions of jobs 073–081, 27 held; job 082 scored 2 of 5, with one not scorable. Job 084 ran on a throttled card and was rerun, with its predictions unchanged, as 084b/084c.

Job	Prediction	Outcome
073	Ours+FETCH ahead of FreeToken by $\geq 5\%$ at 11/25%, within $\pm 5\%$ at 40% (gpt-oss)	Failed: +10.9, +8.4%, but -5.9% at 40%
073	Ours+FETCH $\geq 1.8\times$ llama.cpp at every budget	Held: 1.83 / 2.36 / $2.65\times$
073	Warm-up on the measured problem reads 5–15% faster	Failed: -0.9% (ours), -0.3% (FreeToken)
073	$\geq 80\%$ of greedy outputs identical to llama.cpp’s	Failed: 27–47%; kernels round differently
074	Host launch ≤ 0.25 ms	Held
074	40%: ≥ 127 tok/s and ahead of FreeToken	Failed: 95.1 vs 114.4
074	≥ 59 / ≥ 91 tok/s at 11 / 25%	Failed: 41.5, 65.3 (host memory 33% slower than assumed)
074	Admission half-life 64 within $\pm 2\%$	Held
074b	GPU sampling cuts between-step time by ≥ 1 ms	Held: 2.01→0.42 ms

Job	Prediction	Outcome
074b	Ours gains $\geq 10\%$ at 40%	Held: +11.7%
074b	With FETCH and GPU sampling, ours beats FreeToken offload at 40%	Held
075	40% (RTX PRO 6000): ours ahead of FreeToken, CI > 1	Held: +12.2%
075	60%: ours within -3 to $+10\%$ of FreeToken	Failed, favourably: +11.8%
075	60%: ours $\geq 55\%$ of all-in-VRAM speed	Held: 72.7%
076	Ours ahead of FreeToken at 11/25/40%	Held: 1.21 / 1.22 / 1.12
076	Ours $\geq 1.8\times$ llama.cpp	Held
076	The law within 8% on the machine	Held: $-0.9, -0.6, -5.0\%$, FETCH +4.1%
077	Pipelined Sharding and leloch’s cache run and beat stock llama.cpp	Held
077	Ours ahead of both at every budget	Held
078	Qwen3: ours ahead of FreeToken at 12.5 and 25%	Held: +7%
078	Ours ahead of KTransformers at every budget	Held: 2.2–4.4 $\times$
078	Ours $\geq 1.5\times$ llama.cpp	Held: 2.1 / 3.1 / 4.5 $\times$
079	Our host-side time per step is small	Held: 0.27 ms
079	We launch more kernels than FreeToken	Held: 1.60 $\times$
079	Parity with FreeToken at 43.75% on this host	Held: 0.991 [0.975, 1.009]
080	Law table beats fixed by $\geq 3\%$ at 43.75%	Held: +4.8%
080	Law table within 2% of the sweep’s best	Held: 0.9%
080	Best table leads FreeToken at 43.75%	Held: +4.9%
080	Law table beats fixed at 25%	Held: +6.7%
081	Law table beats fixed on gpt-oss at every budget	Held: +7.9, +5.4, +3.3%
081	Ours leads FreeToken on gpt-oss at 11/25/40%	Held
081	Qwen3 12.5%: law beats fixed; ours leads FreeToken	Held: +8.0%; +3.2%
081	Ours $\geq 1.8\times$ llama.cpp (gpt-oss), $\geq 2\times$ (Qwen3 12.5%)	Held
082	gpt-oss steady-state lead at 2,048 tokens	Not scorable: easy prompts ended early
082	FreeToken gains more than ours after 256 tokens	Failed: ours gained more
082	Qwen3 43.75% steady state: FreeToken ahead	Failed, favourably: ours +5.4%
082	Every ablation step $\geq -1\%$; ours $\geq 2\times$ stock	Failed: the “static” step, 0.72 / 0.74 $\times$, loaded no experts (rerun as 087)
082	Parity: $\geq 98\%$ top-1, $ \Delta\text{NLL} \leq 1\%$	Held
082	llama.cpp Qwen3 within 5% of the law	Held: +3.6, +4.0%
083	Hard problems still reasoning at 2,048 tokens on ≥ 8 of 10	Held: 9–10 of 10
083	Ours within 15% of its Table 1 rate on long outputs	Held: +3.3 to +4.6%
083	Ours leads FreeToken at 11/25/40% (CI > 1)	Held: 1.36 / 1.32 / 1.19
083	Lead $\geq 15\%$ at 11 and 25%	Held
084	Qwen3 BF16 all in VRAM at 120–190 tok/s	Held (084b): 177.8
084	gpt-oss all in VRAM within 5% of 261.4 tok/s	Held (084b): 255.3
084	Ours with every expert in slots within 5% of stock all-in-VRAM	Held: 0.997, 1.047
084	Trace-simulated hit rate within 5 points of the engine’s; ours at 20–50% of the limit	Half: 5.3 points off at Qwen3 12.5%; 26–43%
085	The law’s tables fetch less than on the headline machine	[pend.]
085	Law beats fixed at every budget, by more than on the headline machine	[pend.]
085	Ours leads FreeToken at every budget	[pend.]
085	Ours’ lead larger than on the headline machine at ≥ 4 of 6 budgets	[pend.]
085	llama.cpp at 25% within 5% of the law	[pend.]
086	FreeToken as shipped within 15% of its paper’s 77–83 tok/s	[pend.]
086	Ours leads FreeToken at 12.5/25/37.5%	[pend.]

Job	Prediction	Outcome
086	Ours $\geq 2\times$ llama.cpp	[pend.]
086	Law table beats fixed at every budget	[pend.]

C Artifact description

- **System.** A patch to llama.cpp commit `4da6337` (the expert cache, FETCH, the mailbox and the `llama-ec-bench` instrumentation: teacher-forced loss, logit dumps, routing lookahead records); built with CUDA 13.0 for compute capability 12.0.
- **Jobs.** One shell script per job; each installs FreeToken at commit `0d652e7`, downloads the models from Hugging Face, runs the bandwidth probes, writes any on-machine prediction, runs the client, and stores raw rows (one JSON line per problem), server logs and engine counters. A small runner rents a Vast instance, streams a heartbeat, fetches the results and only then destroys the instance.
- **Analysis.** `scripts/*_stats.py` compute every interval from the raw rows; `scripts/wsg_tables.py` generates every table, figure and number in this paper from the outcome files.
- **Cost.** [pend.] (Vast balance at submission).